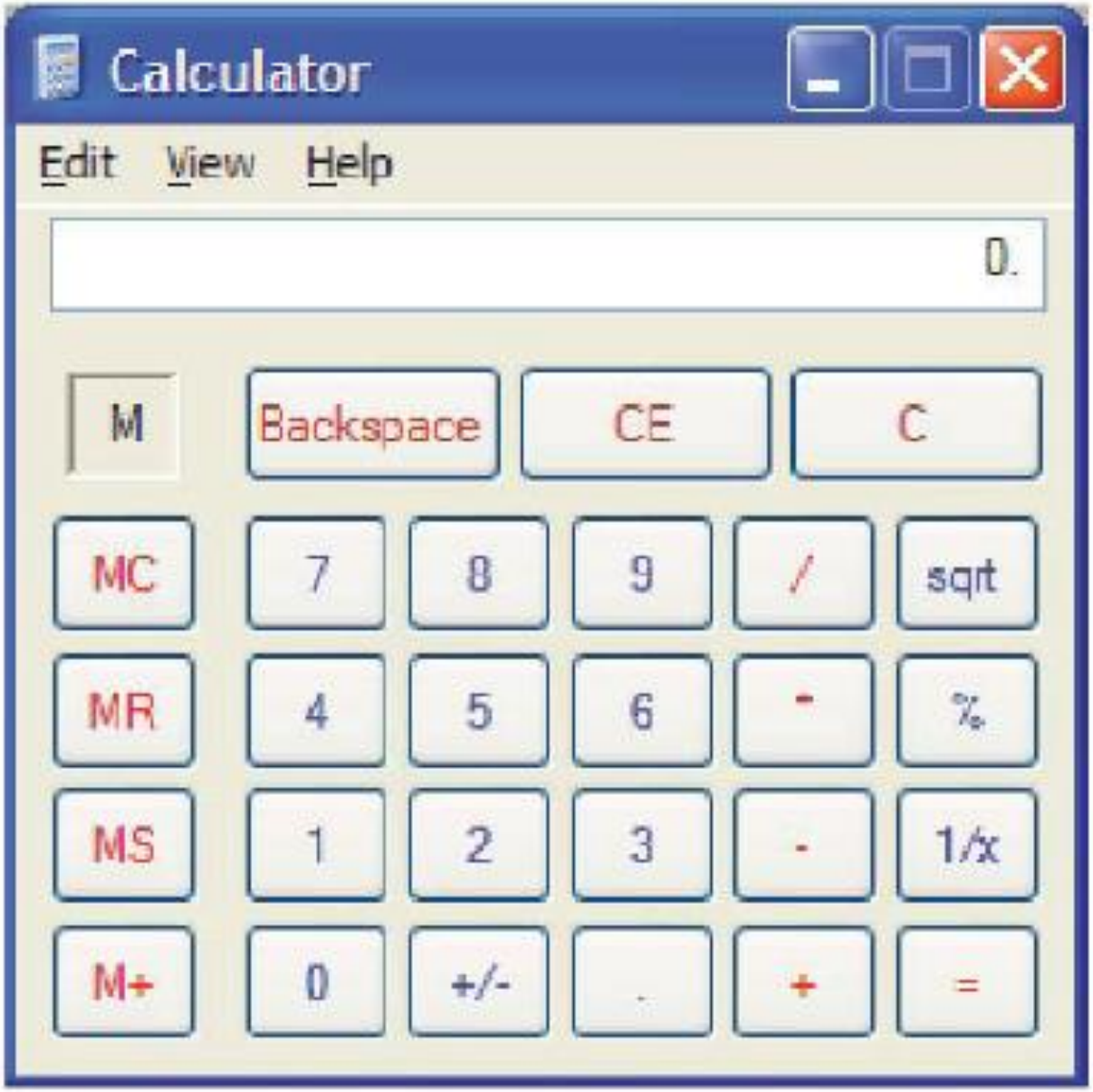
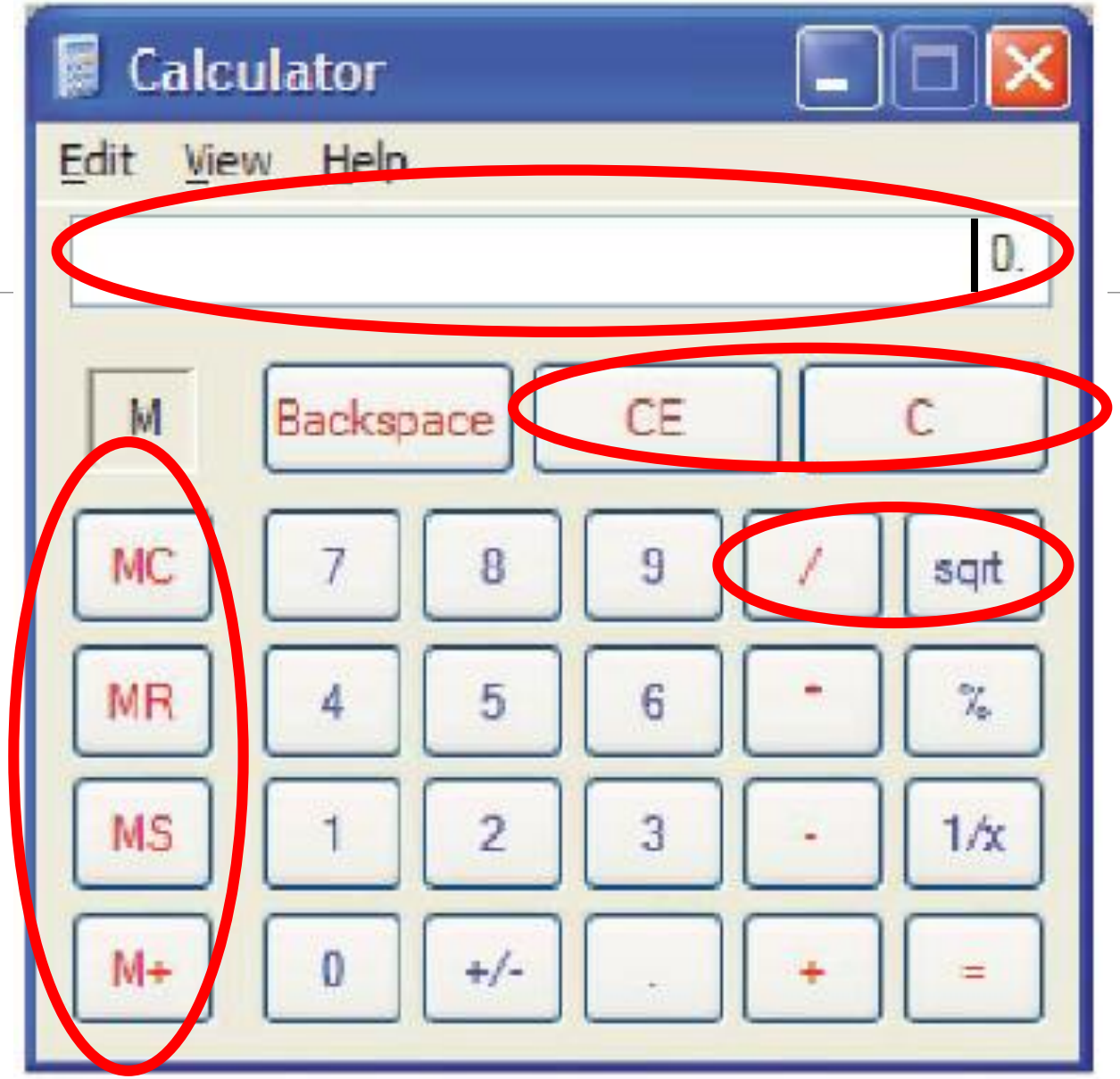
Errors & User Control


Calculator
Edit View Help
0.
M
Backspace
CE
C
MC
7
8
9
/
sqrt
MR
4
5
6
-
%
MS
1
2
3
-
1/x
M+
0
+/-
.
+
=



given an onscreen calculator like this one to use, 13 of 24 people never realized that they could use the keyboard

Today's Topics

- Human error
- Design principles
  - Error prevention
  - Error messages
  - User control & freedom
- Undo

Error Types

- Slips and lapses
  - Failure to correctly execute a procedure
  - Slip is a failure of execution, lapse is a failure of memory
  - Typically found in skilled behavior
- Mistakes
  - Using wrong procedure for the goal
  - Typically found in rule-based behavior or problem-solving behavior

Slips

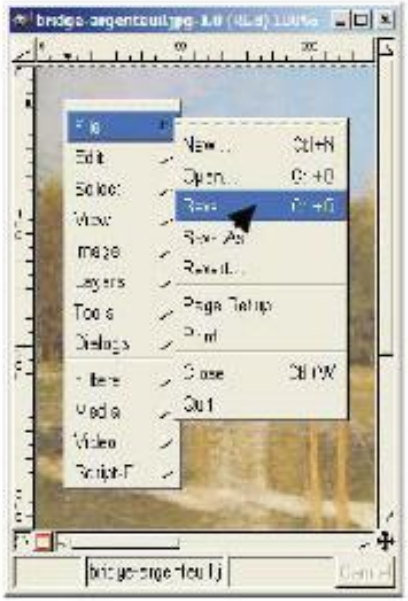
- Capture
  - Leave your house and find yourself walking to school instead of where you meant to go
  - vi :w command (to save the file) vs. :wq command (to save and quit)
  - Excel array formulas must be entered with Ctrl-Shift-Enter, not just Enter
- Description
  - Putting the wrong lid on a bowl
  - Throwing shirt into toilet instead of hamper
  - Choosing Kendall Square instead of Kenmore Square

Lapses

- Loss of intention
  - Walking to another room and forgetting why you went there
- Omissions due to interruption
  - Getting coat to go out, then interrupted by a phone call; then go out without your coat
- Omissions due to already-satisfied goal
  - Walking away from an ATM without your card
  - Walking away from a copier without your originals

Mode Error

- Modes: states in which actions have different meanings
  - Vi's insert mode vs. command mode
  - Caps Lock
  - Drawing palette



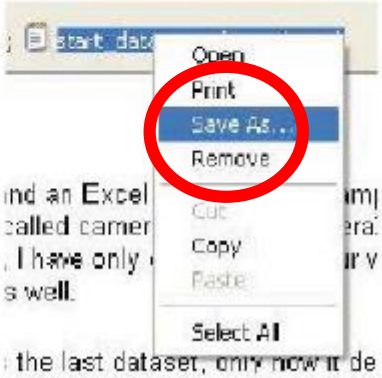
Mode errors are generally slips, an error in the execution of a learned procedure, caused by failing to correctly evaluate the state of the interface

Common Features of Human Error

- Inattention or inappropriate attention
  - Causes slips and lapses, but not mistakes
- "Strong-but-wrong" effect
  - Similarity
  - High frequency

Avoiding Capture and Description Slips

- Avoid habitual action sequences with identical prefixes
- Avoid actions with very similar descriptions
- Keep dangerous commands away from common ones

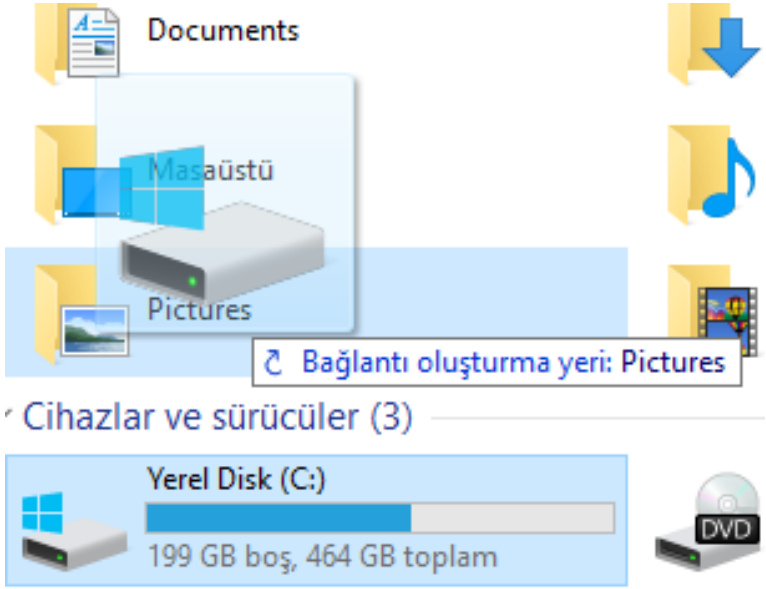


Outlook 2003

© Microsoft. All rights reserved.

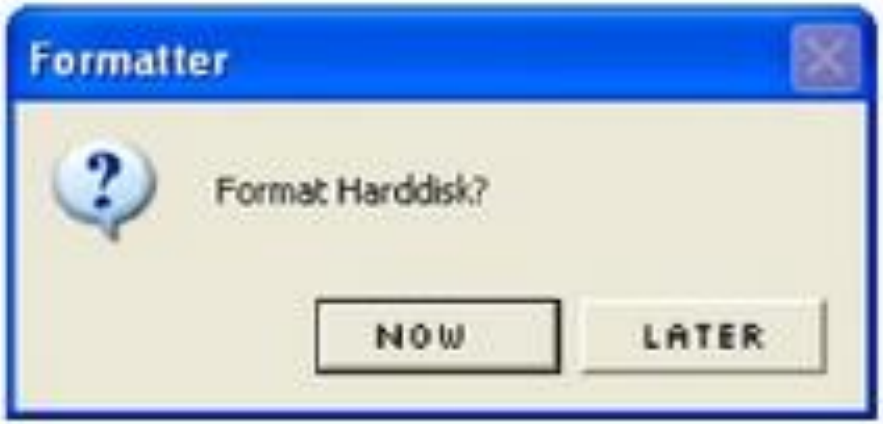
Avoiding Mode Errors

- Eliminate modes
- Increase visibility of mode
- Spring-loaded or temporary modes
- Disjoint action sets in different modes



Avoiding Lapses

- Keep procedures short
  - Provide dialog closure
- Minimize interruptions
- Use forcing functions
  - In automatic transmission, you must hold down the brake in order to shift out of Park
  - Must take your ATM card out of the machine before you get your money



key feature of closure is the feedback you get at the end of the operation. assurance that the operation completed provides the user with a sense of accomplishment, some relief, and an opportunity to clear their working memory of the details of the task

Other Rules for Error Prevention

- Disable illegal commands
- Use menus & forms, not command languages
- All needed information should be visible
- Use combo boxes, not textboxes
  - But don't go overboard...

Source: Interface Hall of Shame

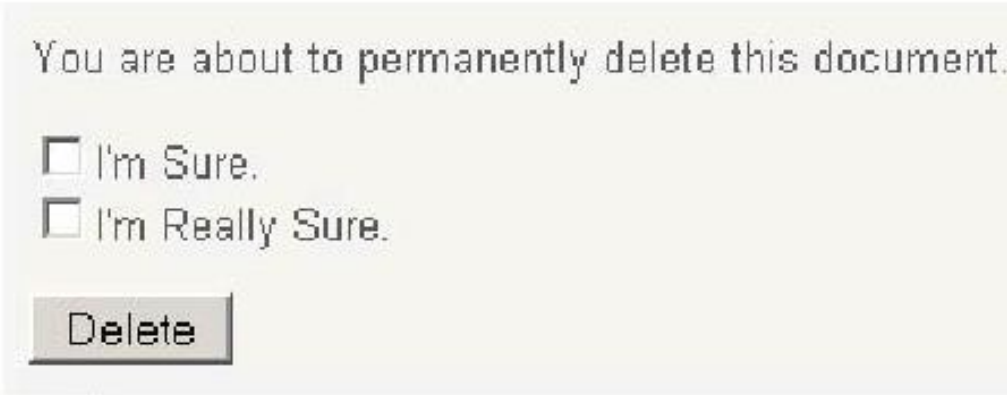
© Interface Hall of Shame. All rights reserved.

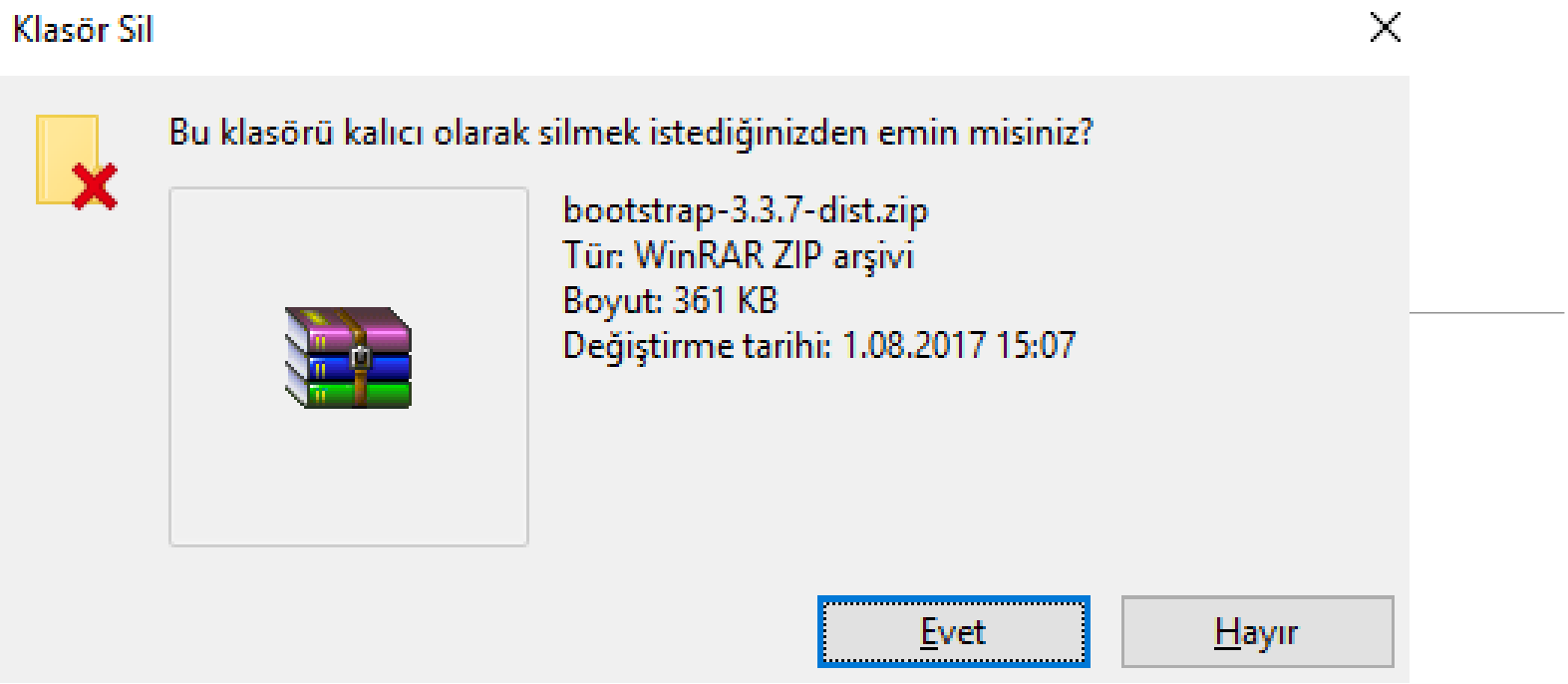
- Protect user's work

automatic save, undo, file version histories, etc.

Confirmation dialog

Efficiency?



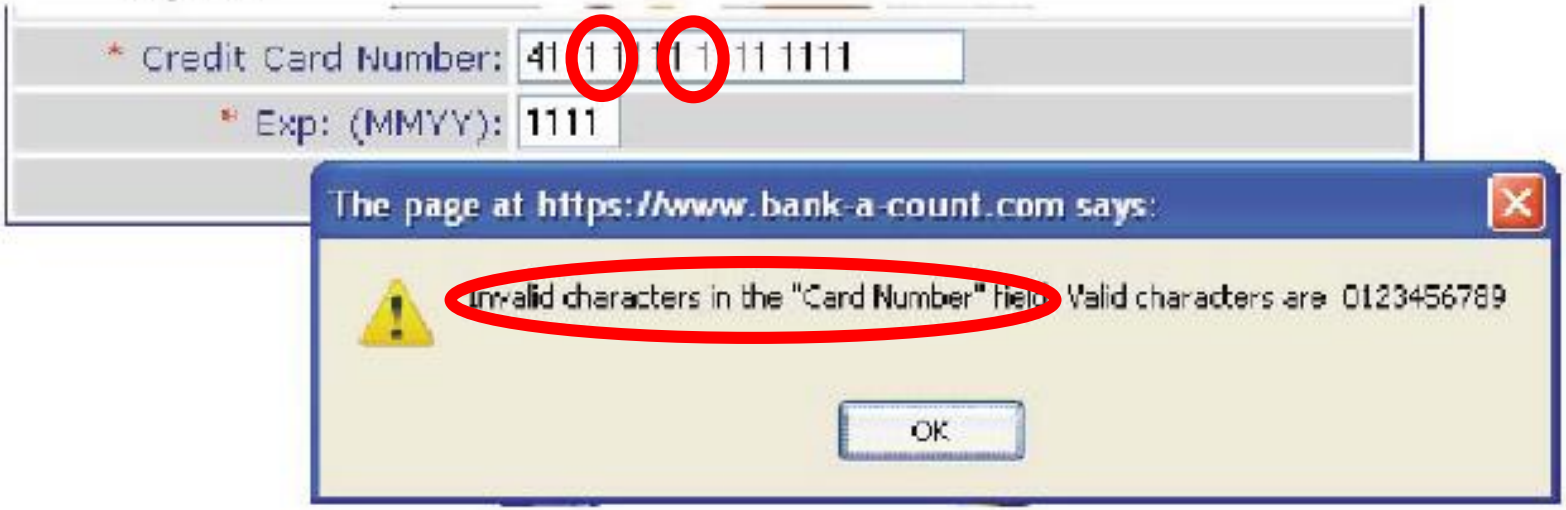


A frequent user will combine Shift+Delete+Enter

Dialog will just decrease the efficiency without preventing errors

Writing Error Message Dialogs

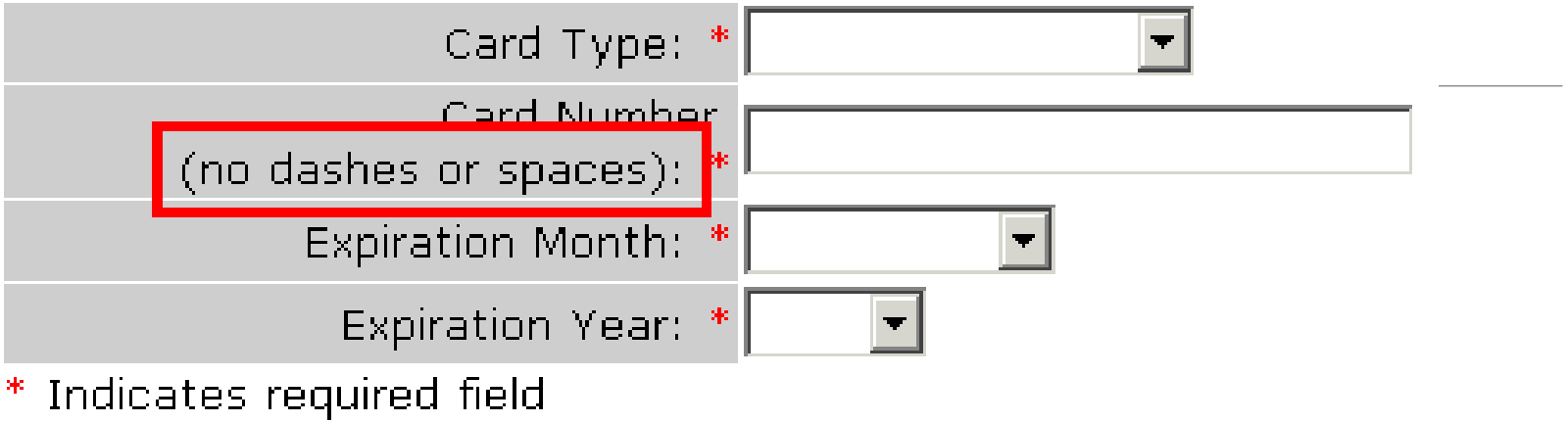
- Best error message is **none at all**
  - Errors should be prevented
  - Be more flexible and tolerant
  - Nonsense entries can often be ignored without harm



Source: "No Dashes Or Spaces" Hall of Shame

if the user drags a scrollbar thumb too far, the scrollbar doesn't pop up an error message ("Too far! Too far!"). It simply stops.

No Dashes or Spaces Hall of Shame (http://www.unixwiz.net/ndos-shame.html)



It turns out that one can clean up the spaces and dashes in approximately the same amount of code as the instruction not to (example in Matlab):

```
strrep(strrep(a,' ',''),'-','')   % (No dashes or spaces)
```

```
a='1 2- 3---';
strrep(strrep(a, ' ', ''), '-', '')
ans = 123
```

Why are dashes and spaces helpful for human perception and memory?
Chunks

Create Password: ●●●●●●●●●●●●

Please enter numbers here.

Prevent this page from creating additional dialogs

OK

Payment (Your credit card info is not stored on Eventbrite

Credit Card: * MasterCard

Credit Card Number: * 5111 SPACE

Credit Card (details)

Please enter your credit card as a continuous string of numbers without spaces or dashes.

The security of your credit card information is important to us. We use Secure Socket Layer (SSL) t

Credit Card Number:

Expiration Date: Select One / Select One

Be Precise and Comprehensible

- Be precise
  - "File missing or wrong format"
  - "File can't be parsed"
  - "Line too long"
  - "Name contains bad characters"
- Restate user's input
  - Not "Cannot open file", but "Cannot open file named paper.doc"
- Speak the user's language
  - Not "FileNotFoundException"
  - Hide technical details (like a stack trace) until requested

Suggest Reasons and Solutions

- Give **constructive** help
  - why error occurred and how to fix it

© Adobe Systems Inc. All rights reserved.

Be Polite

- Be polite and nonblaming

- Avoid loaded words

Fatal, illegal, aborted, terminated

© Microsoft. All rights reserved.

AutoCad - it was inserted by a programmer as a joke, but somehow never removed before release

User Control & Freedom

- Learning by exploring
- Dealing with errors
- Long operations should be cancelable

© source unknown. All rights reserved.

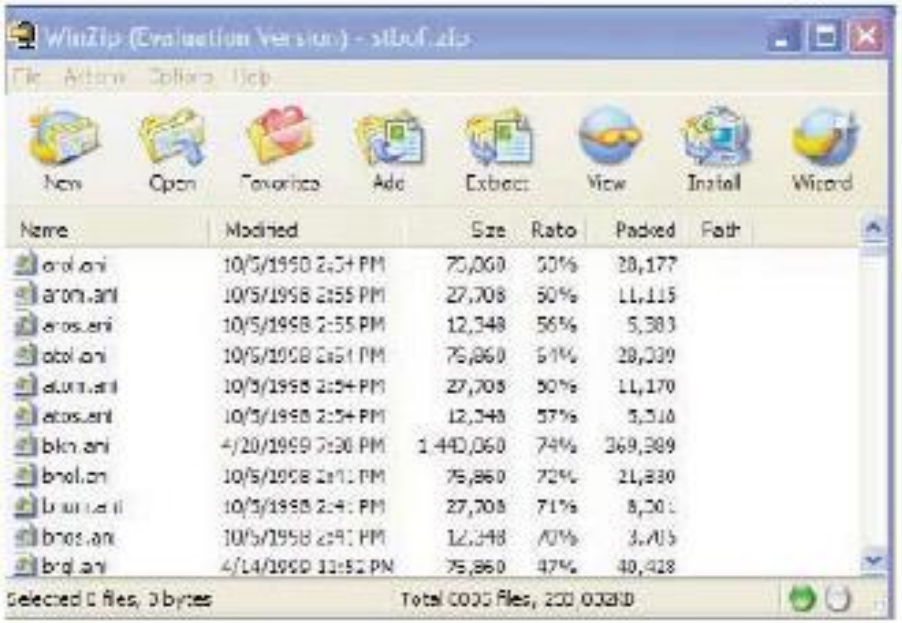
- All dialogs should have a cancel button

Wizard

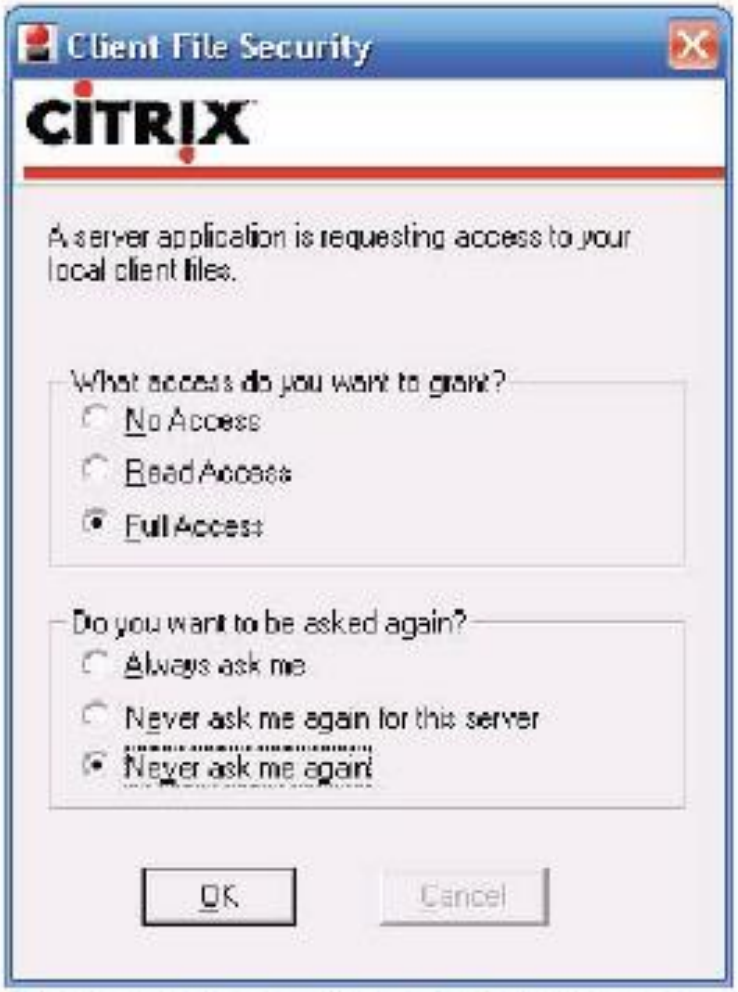
Imagine a travel agent who's asking you a series of questions, and refuses to listen to what you say if it's not relevant to the question they asked. That's a wizard.

Wizards clearly restrict the user's freedom, but for complex, infrequently-done tasks (like installation), the tradeoff is often worth it.

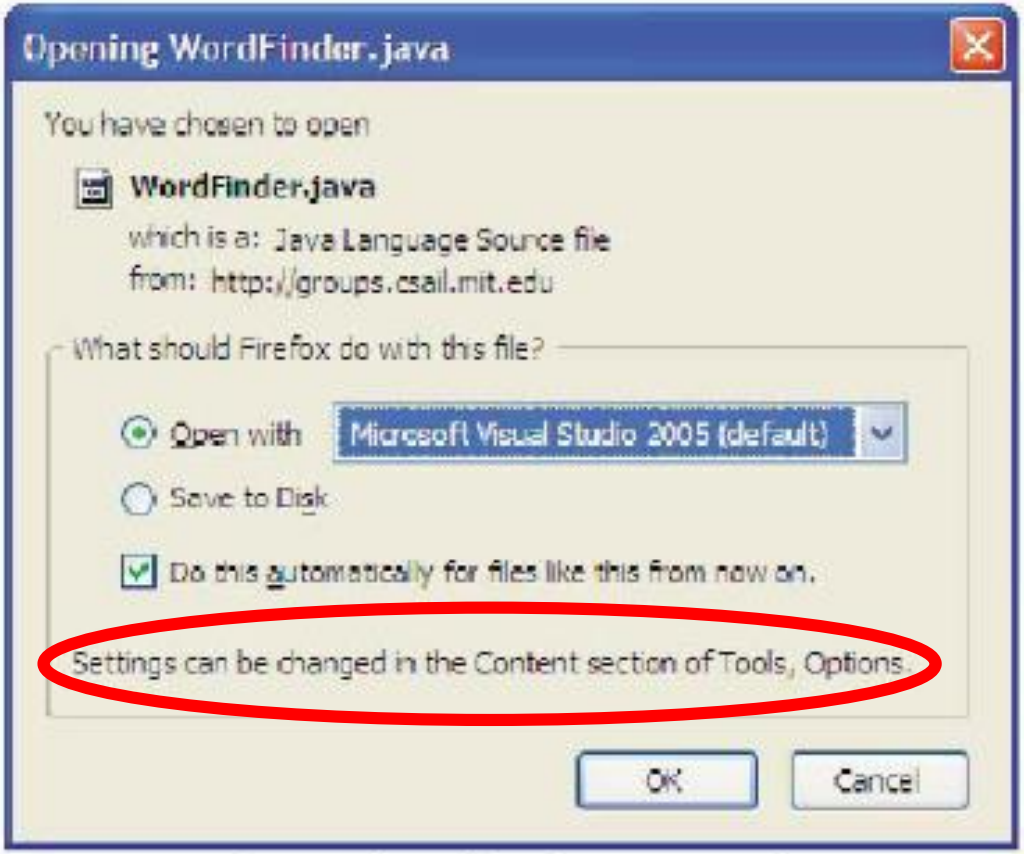
Center Stage



Never Ask Me Again



© Citrix Systems, Inc. All rights reserved.



© Mozilla. All rights reserved.

just because the user has seen a message doesn't mean they've *learned* what it had to say

User Control Over Data

- Data entered by the user should be editable by the user
- UI should give the power to:
  - Create a data item
  - Read it
  - Update it
  - Delete it

No Arbitrary Limits on User-Defined Names

© Facebook. All rights reserved.

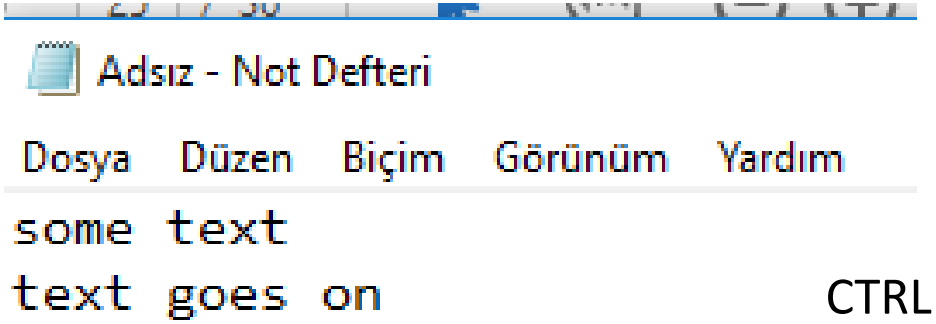
Support Undo

- Desktop

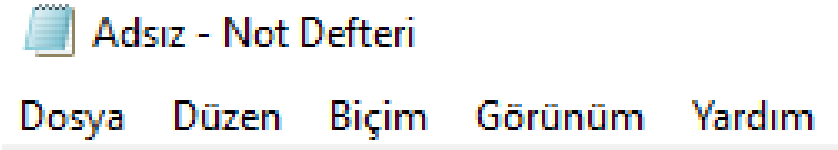
© Apple Inc. All rights reserved.

- Web

© Google. All rights reserved.



CTRL + Z



Forming a Mental Model of Undo

- Undo reverses the effect of an action
- But that leaves many questions:
  - What stream of actions will be undone?
  - How is the stream divided into undoable units?
  - Which actions are undoable, and which are skipped?
  - How much of the previous state is actually recovered by the undo?
  - How far back in the stream can you undo?

What stream of actions will be undone?

- Actions in this window (MS Office)
- Actions in this text widget (web browser)
- Just my actions, or everybody's (multiuser apps)
- Actions made by the computer
  - MS Office AutoCorrect and AutoFormat are undoable, even though user didn't do them

How is the stream divided into units?

- Lexical level
  - Mouse clicks, key presses, mouse moves
  - Nobody does it at this level
- Syntactic level
  - Commands and button presses
- Semantic level
  - Changes to application data structures (e.g., the result of an entire Format dialog)
  - This is the normal level
- Text entry is aggregated into a single action
  - But other editing commands (like Backspace) and newlines interrupt the aggregation
- What about user-defined macros?
  - Undo macro actions individually, or as a unit?

Which actions are undoable?

- User's action stream may include many actions that are ignored by Undo
  - Selection
  - Keyboard focus
  - Changing viewpoint (scrolling, zooming)
  - Changing layout (opening palettes or sidebars, adjusting window sizes)
  - UI customization (adding buttons to toolbars)

How much state is recovered?

- Select text, delete it, and then undo
  - Text is restored
  - But is selection restored? Cursor position?

How far back can you undo?

- Often a limit on history size
  - Used to be one action -- now usually hundreds, or infinite
- Does action stream persist across application sessions?
  - If so, stream must be saved to file
- Does it persist across File/Save?

Design Principles for Undo

- Visibility
  - Make sure undone effects are visible
    - e.g., scrolled into view, selected, possibly animated
- Aggregation
  - Units should be "chunks" of action stream: typed strings, dialogs, macros
- Reversibility of the Undo itself
  - Support Redo as well as Undo
  - Undo to a state where user can immediately reissue the undone command, or a variant on it
    - e.g., restore selection & cursor position
- Reserve it for model changes, not view changes
  - For consistency with other applications, reserve Undo for changes to backend data
- "Undo" is not the only way to support reversibility
  - Backspace undoes typing, Back undoes browsing, Recent Files undoes file closing, scrolling back undoes scrolling
  - Forward error recovery: using new actions to fix errors

Summary

- Human error
  - Errors include slips, lapses, mistakes
  - Human error is characterized by inattention, similarity, frequency
- Design principles
  - Prevent errors as much as possible
  - Write good error messages otherwise
  - Give user control over the dialog
    - At least veto power: every operation should have Cancel
  - Give user control over data
    - CRUD: Create, Read, Update, Delete
- Support undo
  - But it's more complex than it seems